

02_staging_intermediate

3. Staging Models

Staging is the **renaming layer**, and nothing more. Each staging model is a thin, mechanical, **1:1 projection of exactly one source object** whose only jobs are: rename to `snake_case`, cast to the right type, parse the shard suffix into a real `event_date`, and perform *light* flattening of GA4's nested-but-scalar structs (`device.*`, `geo.*`, `ecommerce.*`). The discipline that makes the rest of the project tractable is the list of things staging **must not** do:

- **No joins.** A staging model touches one source and only that source. The moment you join, you have business logic, and business logic lives downstream.
- **No aggregations.** Staging preserves the source grain. `stg_ga4__events` is one row per event; `stg_ga4__event_params` is one row per (event, param key) . No `GROUP BY`, no window collapsing.
- **No deduplication or filtering of business rows.** Staging keeps every event. Sessionization, funnel logic, and "engaged session" rules are intermediate concerns.
- **No `SELECT *`.** Every column is named explicitly. This is a BigQuery cost rule (column pruning) *and* a contract: a new column in the GA4 export can never silently leak into marts.

Because staging is pure projection it is materialized as a **view** — zero storage cost, always reflects the latest source, and the renamed/typed shape is computed at query time. The two staging models are the *only* place in the entire project that references the raw `event_params ARRAY<STRUCT>` and the GA4 nested structs directly. Everything downstream consumes the clean, typed columns and the long param table, so we never re-implement `UNNEST` or remember which key is an int vs. a string twice.

3.1 Source declaration (`src_ga4`)

Staging reads through `source('src_ga4', 'events')`, never a hard-coded table name. The source pins the date-sharded `events_*` wildcard, the partition expectation, and **freshness**. Honesty note: the public `bigquery-public-data.ga4-obfuscated-sample-ecommerce` sample is **static and historical** (2020-11-01..2021-01-31, no live updates), so a real freshness check is **N/A** on it — `dbt source freshness` will always report the data as stale because the newest shard is years old. We therefore design the **production** freshness contract (a live daily GA4 export should land `events_YYYYMMDD` within ~36h) and **disable the freshness error on the sample** so the build is honest rather than red for a reason that doesn't apply.

```

# models/staging/src_ga4.yml
version: 2

sources:
  - name: src_ga4
    database: bigquery-public-data
    schema: ga4_obfuscated_sample_ecommerce
    description: >
      Raw GA4 BigQuery export (date-sharded events_YYYYMMDD). The PUBLIC SAMPLE
      is static/historical (2020-11-01..2021-01-31); in PRODUCTION this is the
      live daily export landing one shard per day.
    # PRODUCTION freshness SLA. On the static sample this always trips because
    # the newest shard is historical, so we override loaded_at_field below and
    # rely on a date-bounded singular test instead of dbt source freshness.
    freshness:
      warn_after: {count: 36, period: hour}
      error_after: {count: 48, period: hour}
    # _TABLE_SUFFIX is 'YYYYMMDD'; parse it to a timestamp for freshness.
    loaded_at_field: "parse_timestamp('%Y%m%d', _table_suffix)"
    tables:
      - name: events
        identifier: "events_*" # wildcard over the date shards
        description: One row per GA4 event; partitioned/sharded by event date.
        # On the sample, override to skip freshness (data is intentionally old):
        # freshness: null
        loaded_at_field: "parse_timestamp('%Y%m%d', _table_suffix)"

```

3.2 stg_ga4__events.sql

One row per event, typed and renamed, with the scalar session/device/geo/ecommerce fields surfaced and `session_key` computed once via the `sessionize()` macro. This is where `_TABLE_SUFFIX` becomes a real `event_date` (the partition column for every downstream incremental fact), and where the `is_incremental()` lookback prunes shards so a daily run scans three shards, not ninety.

```

-- models/staging/stg_ga4__events.sql
{{ config(materialized='view') }}

with source as (

  select
    -- shard suffix 'YYYYMMDD' → a real DATE partition key
    parse_date('%Y%m%d', _table_suffix) as event_date,
    _table_suffix as table_suffix,

    -- identity & event
    event_name as event_name,
    timestamp_micros(event_timestamp) as event_timestamp,
    user_pseudo_id as user_pseudo_id,

```

```

user_id                                as user_id,          -- null on this
cookie-grain sample

-- session id lives in event_params, not a top-level column
{{ get_event_param('ga_session_id', 'int') }} as ga_session_id,
{{ get_event_param('ga_session_number', 'int') }} as ga_session_number,
{{ get_event_param('page_location', 'string') }} as page_location,
{{ get_event_param('session_engaged', 'string') }} as session_engaged,
{{ get_event_param('engagement_time_msec', 'int') }} as engagement_time_msec,

-- SESSION-SCOPED source/medium/campaign (see traffic_source gotcha in 4.1)
{{ get_event_param('source', 'string') }} as event_source,
{{ get_event_param('medium', 'string') }} as event_medium,
{{ get_event_param('campaign', 'string') }} as event_campaign,
{{ get_event_param('gclid', 'string') }} as gclid,

-- USER first-touch attribution (FALLBACK only)
traffic_source.source as first_touch_source,
traffic_source.medium as first_touch_medium,
traffic_source.name as first_touch_campaign,

-- device.* (light struct flattening; no joins)
device.category as device_category,
device.operating_system as operating_system,
device.web_info.browser as browser,
device.language as device_language,

-- geo.*
geo.country as country,
geo.region as region,
geo.city as city,

-- ecommerce.* (purchase rows; usd only per money convention)
ecommerce.transaction_id as transaction_id,
ecommerce.purchase_revenue_in_usd as purchase_revenue_in_usd,
ecommerce.refund_value_in_usd as refund_value_in_usd,
ecommerce.shipping_value_in_usd as shipping_value_in_usd,
ecommerce.tax_value_in_usd as tax_value_in_usd,
ecommerce.total_item_quantity as total_item_quantity

from {{ source('src_ga4', 'events') }}

where 1 = 1
-- PRODUCTION incremental note: although this view is not itself incremental,
-- it is the upstream of incremental facts. Downstream incrementals prune the
-- shard scan with a 3-day lookback; to bound the view's own scan you can add:
-- {% if target.name == 'prod' %}
--     and parse_date('%Y%m%d', _table_suffix)
--     between date '2020-11-01' and date '2021-01-31' -- sample window
-- {% endif %}
-- On a LIVE export, replace the fixed window with a relative bound, e.g.
-- and _table_suffix >= format_date('%Y%m%d', date_sub(current_date(), interval 3 day))
-- so a daily run scans ~3 shards. Prune via _TABLE_SUFFIX, never SELECT *.

```

```

),

final as (

    select
        {{ sessionize() }}                                as session_key,
        *
    from source

)

select * from final

```

`session_key` is built **only** through `sessionize()` so the canonical expression `TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))` exists in exactly one place. The `select *` in `final` is over an already-explicit CTE (every column was named in `source`), so the no-`SELECT *`-against-source rule still holds.

3.3 `stg_ga4__event_params.sql`

GA4 stores `event_params` as an `ARRAY<STRUCT<key STRING, value STRUCT< ... >>>`. Re-`UNNEST`-ing it in every model that needs a param is both error-prone and expensive. This staging model flattens it **once** into a long key/value table at `(event x param key)` grain so downstream code can `where key = ' ... '` instead of writing `UNNEST` and remembering which of `string_value` / `int_value` / `float_value` / `double_value` is populated.

```

-- models/staging/stg_ga4__event_params.sql
{{ config(materialized='view') }}

with events as (

    select
        parse_date('%Y%m%d', _table_suffix) as event_date,
        event_name,
        event_timestamp,
        user_pseudo_id,
        event_params                                -- the raw ARRAY<STRUCT> column
    from {{ source('src_ga4', 'events') }}

),

unnested as (

    select
        e.event_date,
        e.event_name,
        timestamp_micros(e.event_timestamp) as event_timestamp,
        e.user_pseudo_id,

```

```

    ep.key                                as param_key,
    ep.value.string_value                 as string_value,
    ep.value.int_value                    as int_value,
    ep.value.float_value                  as float_value,
    ep.value.double_value                 as double_value,
    -- a single coalesced text projection for convenience downstream
    coalesce(
        ep.value.string_value,
        cast(ep.value.int_value as string),
        cast(ep.value.float_value as string),
        cast(ep.value.double_value as string)
    )                                     as value_string
from events e,
     unnest(e.event_params) as ep
)

select * from unnested

```

3.4 The three macros

These are the only abstractions staging and intermediate share. Each is a **single source of truth**.

get_event_param(key, type) — typed extraction from the `event_params` array. It generates the `(SELECT ... FROM UNNEST(event_params) WHERE key = ...)` correlated subquery and selects the right value slot for the requested type, so models never hand-write `UNNEST` for a single key.

```

-- macros/get_event_param.sql
{% macro get_event_param(key, type='string') %}
    {% set slot = {
        'string': 'string_value',
        'int':    'int_value',
        'float':  'float_value',
        'double': 'double_value'
    } -%}
    (
        select ep.value.{{ slot[type] }}
        from unnest(event_params) as ep
        where ep.key = '{{ key }}'
        limit 1
    )
{% endmacro %}

```

sessionize() — the canonical session-key expression, defined once. `sessions = COUNT(DISTINCT session_key)` everywhere; the key is never `FARM_FINGERPRINT` and never `COUNT(*)`.

```
-- macros/sessionize.sql
{% macro sessionize(user_col='user_pseudo_id', session_id_col='ga_session_id') %}
  to_hex(md5(concat(
    {{ user_col }}, '-', cast({{ session_id_col }} as string)
  )))
{% endmacro %}
```

channel_group() / **channel_group_case()** — the **single source of truth** for the 10 GA4 default channel groups. Every model and the semantic layer derive **channel_group** from this macro; there is no 11th group and no "Paid Other". It is **has_gclid**-aware (a **gclid** forces Paid Search even when the medium is dirty) and applies GA4's default-channel-grouping precedence.

```
-- macros/channel_group.sql
{% macro channel_group(source_col='source', medium_col='medium', gclid_col='gclid') %}
  {{ return(channel_group_case(source_col, medium_col, gclid_col)) }}
{% endmacro %}

{% macro channel_group_case(source_col='source', medium_col='medium', gclid_col='gclid') %}
  case
    -- Direct: no/(direct)/(none) source and (none)/(not set) medium
    when ( {{ source_col }} is null or lower({{ source_col }}) in ('(direct)', 'direct') )
      and ( {{ medium_col }} is null or lower({{ medium_col }}) in ('(none)', '(not set)') )
      then 'Direct'

    -- Paid Search: gclid present, OR known search source on a paid medium
    when {{ gclid_col }} is not null
      or ( regexp_contains(lower({{ source_col }}),
        r'google|bing|yahoo|duckduckgo|baidu|yandex|ecosia')
        and regexp_contains(lower({{ medium_col }}), r'^(.cp.*|ppc|paid|retargeting)$') )
      then 'Paid Search'

    -- Paid Social: known social source on a paid medium
    when regexp_contains(lower({{ source_col }}),
      r'facebook|instagram|tiktok|twitter|x\.com|linkedin|pinterest|reddit|snapchat')
      and regexp_contains(lower({{ medium_col }}), r'^(.cp.*|ppc|paid|social-
        paid|retargeting)$')
      then 'Paid Social'

    -- Display: banner/display/cpm/expandable/interstitial mediums
    when regexp_contains(lower({{ medium_col }}),
      r'^(display|banner|cpm|expandable|interstitial)$')
      then 'Display'

    -- Organic Search: search engines on organic medium
    when regexp_contains(lower({{ source_col }}),
      r'google|bing|yahoo|duckduckgo|baidu|yandex|ecosia')
      and lower({{ medium_col }}) = 'organic'
      then 'Organic Search'

    -- Organic Social: social sources, organic/social/referral medium
    when regexp_contains(lower({{ source_col }}),
      r'facebook|instagram|tiktok|twitter|x\.com|linkedin|pinterest|reddit|snapchat|youtube')
```

```

    and lower({{ medium_col }}) in ('social', 'social-network', 'social-
media', 'sm', 'organic', 'referral')
    then 'Organic Social'

-- Email
when lower({{ source_col }}) = 'email'
    or regexp_contains(lower({{ medium_col }}), r'^e?-?mail$')
    then 'Email'

-- Affiliates
when lower({{ medium_col }}) = 'affiliate'
    then 'Affiliates'

-- Referral: any explicit referral medium not caught above
when lower({{ medium_col }}) in ('referral', 'link')
    then 'Referral'

-- Everything else
else 'Other'
end
{% endmacro %}

```

3.5 Staging `schema.yml`

Staging tests assert the projection didn't break: keys are present, the param key/value pairs are well-formed, and source columns are documented. Heavy reconciliation lives in marts; here we test cheaply and structurally.

```

# models/staging/stg_ga4__schema.yml
version: 2

models:
  - name: stg_ga4__events
    description: "1:1 typed/renamed GA4 events; one row per event. session_key surfaced via
    sessionize()."
    columns:
      - name: session_key
        description: "TO_HEX(MD5(user_pseudo_id || '-' || ga_session_id)). Null only when
        ga_session_id is null."
        data_tests:
          - not_null:
              # GA4 emits a few session-less events (e.g. first_open) with null ga_session_id
              config: {where: "ga_session_id is not null"}
      - name: event_date
        description: "Event date parsed from the _TABLE_SUFFIX shard (DATE). Partition key for
        downstream facts."
        data_tests: [not_null]
      - name: event_timestamp
        description: "Event time (TIMESTAMP, from timestamp_micros)."
        data_tests: [not_null]
      - name: user_pseudo_id

```

```

description: "Device/cookie pseudo id; the de facto user grain (user_id is null on this
sample)."
```

- name: `ga_session_id`

```
description: "GA4 session id within a user, from event_params.ga_session_id."
```
- name: `page_location`

```
description: "Full page URL of the event, from event_params.page_location."
```
- name: `event_source`

```
description: "SESSION-SCOPED source from event_params.source (preferred over
first_touch_source)."
```
- name: `event_medium`

```
description: "SESSION-SCOPED medium from event_params.medium."
```
- name: `device_category`

```
description: "device.category: mobile / desktop / tablet."
```
- name: `country`

```
description: "geo.country."
```
- name: `transaction_id`

```
description: "ecommerce.transaction_id; non-null only on purchase events."
```
- name: `purchase_revenue_in_usd`

```
description: "ecommerce.purchase_revenue_in_usd (USD only, per money convention)."
```

```
data_tests:
```

 - `dbt_utils.accepted_range:`

```
min_value: 0
config: {where: "purchase_revenue_in_usd is not null"}
```

```

- name: stg_ga4_event_params
description: "Fully unnested event_params at (event x param key) grain; the only place
UNNEST(event_params) lives."
columns:
  - name: param_key
    description: "event_params[].key."
    data_tests: [not_null]
  - name: event_timestamp
    description: "Event time (TIMESTAMP)."
```

```
data_tests: [not_null]
```

- name: `user_pseudo_id`

```
description: "Device/cookie pseudo id."
data_tests: [not_null]
```
- name: `value_string`

```
description: "Coalesced string projection of the populated value slot."
```

4. Intermediate Models

Intermediate is where **business logic** lives — the rules that more than one mart needs and that no single mart should own. There are exactly two intermediate models, and both are **keystones**: if they are wrong, every downstream number is *silently* wrong (no error, just a quietly incorrect figure). They are:

- `int_ga4__sessionized` — collapses the event stream to **one row per session**, deriving the session-scoped attributes (landing page, source/medium, channel group, device/geo, engagement, new-vs-returning).
- `int_ga4__funnel_steps` — collapses to **one row per session** carrying the `reached_*` funnel flags and `session_revenue`.

Intermediate models are **never exposed to BI or the semantic layer directly**. They are plumbing: the semantic layer references only marts, and marts (`fct_sessions`, `fct_funnel`, `fct_daily_funnel`, ...) are built *from* these intermediates. Because they are internal and consumed by exactly the few marts that sit on top of them, they are materialized **ephemeral** (inlined as CTEs into their consumers — no storage, no extra scan) or **view** when a model is referenced by several marts and inlining would duplicate the scan. The pinned default is ephemeral.

4.1 `int_ga4__sessionized.sql` (KEYSTONE)

Sessionization groups the event stream by the canonical session key — i.e. by (`user_pseudo_id`, `ga_session_id`) — and derives each session's attributes with window/aggregate functions over its events. The two subtle rules:

1. **landing_page = the earliest page_location**. We take the `page_location` of the event with the minimum `event_timestamp` in the session (`ARRAY_AGG( ... ORDER BY event_timestamp LIMIT 1 )`), not just any non-null value.
2. **The traffic_source gotcha**. Event-level `traffic_source.*` is GA4's **user first-touch** attribution — it is the channel that *originally acquired the user*, identical on every session that user ever has. It is **not** the source of *this* session. Using it would mis-credit returning users' sessions to their original acquisition channel and corrupt every channel-level rate (a Simpson's-paradox-grade error). So we prefer the **session-scoped** `event_params.source/medium` (surfaced as `event_source / event_medium` in staging) and **fall back to traffic_source only when the session-scoped value is null**.

`is_new_user` is derived from `ga_session_number = 1` (the user's first-ever session), and `engaged_session` from the canonical rule `session_engaged = '1' OR engagement_time_msec ≥ 10000`. The `channel_group` is computed via the macro from the resolved session-scoped source/medium and `gcLid`.

```
-- models/intermediate/int_ga4__sessionized.sql
{{ config(materialized='ephemeral') }}

with events as (

  select *
```

```

from {{ ref('stg_ga4__events') }}
where ga_session_id is not null -- drop session-less events (e.g. first_open)

),

sessionized as (

select
    session_key,
    user_pseudo_id,
    ga_session_id,

    -- session timing
    min(event_date)                as session_date,
    min(event_timestamp)           as session_start_ts,
    max(event_timestamp)           as session_end_ts,

    -- landing_page = page_location of the EARLIEST event in the session
    array_agg(page_location ignore nulls
               order by event_timestamp limit 1)[safe_offset(0)] as landing_page,

    -- SESSION-SCOPED source/medium with traffic_source first-touch FALLBACK.
    -- event_source/event_medium = event_params.* (session scope, preferred).
    -- first_touch_* = traffic_source.* (USER first-touch, fallback only).
    coalesce(
        max(event_source),
        max(first_touch_source)
    )                                as session_source,
    coalesce(
        max(event_medium),
        max(first_touch_medium)
    )                                as session_medium,
    coalesce(
        max(event_campaign),
        max(first_touch_campaign)
    )                                as session_campaign,
    max(gclid)                       as gclid,

    -- device / geo (stable within a session; max() picks the non-null)
    max(device_category)            as device_category,
    max(operating_system)          as operating_system,
    max(browser)                   as browser,
    max(country)                   as country,
    max(region)                   as region,
    max(city)                      as city,

    -- engagement: canonical rule (≥ , not >)
    max(case
        when session_engaged = '1'
        or engagement_time_msec ≥ 10000 then true
        else false
    end)                            as engaged_session,

```

```

-- new vs returning: first-ever session for this user
max(case when ga_session_number = 1 then true else false end) as is_new_user,
max(ga_session_number) as ga_session_number

from events
group by session_key, user_pseudo_id, ga_session_id

)

select
s.*,
-- channel grouping from the SINGLE SOURCE OF TRUTH macro, on the
-- RESOLVED session-scoped source/medium (+ gclid awareness)
{{ channel_group_case('s.session_source', 's.session_medium', 's.gclid') }} as channel_group
from sessionized s

```

4.2 int_ga4__funnel_steps.sql (KEYSTONE)

The funnel flags are **MAX-DOWNSTREAM monotonic**: `reached_X` is true if the session fired event `X` **or any later stage**. This guarantees by construction that

```

sessions ≥ reached_view_item ≥ reached_add_to_cart ≥ reached_begin_checkout
           ≥ reached_add_shipping_info ≥ reached_add_payment_info ≥ reached_purchase

```

so every step rate is ≤ 1 and the funnel can never widen as it deepens. (The retired `did_*` flags marked only "the session fired exactly this event" and could be non-monotonic — e.g. a session that purchased via a deep link without an explicit `add_to_cart` event would show `did_purchase=1` but `did_add_to_cart=0`, producing a step rate > 1 . `reached_*` fixes this.)

Implementation: per event, compute a boolean "this event is step X or downstream of X", then `LOGICAL_OR` it up to the session grain. `session_revenue` is the session's purchase revenue, **deduped** so a session with two `purchase` events (or a duplicated event) is not double-counted — we take the max purchase revenue tied to the session's transaction rather than summing event rows.

```

-- models/intermediate/int_ga4__funnel_steps.sql
{{ config(materialized='ephemeral') }}

with events as (

  select
    session_key,
    event_name,
    event_date,
    transaction_id,
    purchase_revenue_in_usd

```

```

    from {{ ref('stg_ga4__events') }}
    where ga_session_id is not null

),

-- per-event MAX-DOWNSTREAM membership: a 'view_item' event marks reached_view_item;
-- a 'purchase' event marks ALL of reached_view_item..reached_purchase, etc.
flagged as (

    select
        session_key,
        event_date,
        transaction_id,
        purchase_revenue_in_usd,

        event_name in (
            'view_item', 'add_to_cart', 'begin_checkout',
            'add_shipping_info', 'add_payment_info', 'purchase'
        ) as f_view_item,

        event_name in (
            'add_to_cart', 'begin_checkout',
            'add_shipping_info', 'add_payment_info', 'purchase'
        ) as f_add_to_cart,

        event_name in (
            'begin_checkout', 'add_shipping_info', 'add_payment_info', 'purchase'
        ) as f_begin_checkout,

        event_name in (
            'add_shipping_info', 'add_payment_info', 'purchase'
        ) as f_add_shipping_info,

        event_name in (
            'add_payment_info', 'purchase'
        ) as f_add_payment_info,

        event_name = 'purchase' as f_purchase

    from events

),

session_grain as (

    select
        session_key,
        min(event_date) as session_date,

        logical_or(f_view_item) as reached_view_item,
        logical_or(f_add_to_cart) as reached_add_to_cart,
        logical_or(f_begin_checkout) as reached_begin_checkout,
        logical_or(f_add_shipping_info) as reached_add_shipping_info,

```

```

logical_or(f_add_payment_info) as reached_add_payment_info,
logical_or(f_purchase) as reached_purchase,

-- DEDUPED session revenue: one purchase amount per transaction, summed
-- across distinct transactions in the (rare) multi-order session.
coalesce(sum(distinct_txn_rev), 0) as session_revenue,
max(transaction_id) as transaction_id

from (
  select
    session_key,
    event_date,
    transaction_id,
    f_view_item, f_add_to_cart, f_begin_checkout,
    f_add_shipping_info, f_add_payment_info, f_purchase,
    -- collapse duplicate purchase rows: one revenue value per txn
    max(purchase_revenue_in_usd) over (
      partition by session_key, transaction_id
    ) as distinct_txn_rev
  from flagged
)
group by session_key

)

select * from session_grain

```

4.3 Intermediate `schema.yml`

```

# models/intermediate/int_ga4__schema.yml
version: 2

models:
  - name: int_ga4__sessionized
    description: "KEYSTONE. One row per session; session-scoped source/medium (traffic_source
      fallback), channel_group, engagement, new/returning."
    columns:
      - name: session_key
        data_tests: [not_null, unique]
      - name: channel_group
        description: "From channel_group_case(); one of the 10 canonical groups."
        data_tests:
          - accepted_values:
              values: ['Direct', 'Organic Search', 'Paid Search', 'Display', 'Paid Social',
                'Organic Social', 'Email', 'Affiliates', 'Referral', 'Other']
      - name: engaged_session
        data_tests: [not_null]
      - name: is_new_user
        data_tests: [not_null]

  - name: int_ga4__funnel_steps

```

```

description: "KEYSTONE. One row per session; reached_* MAX-DOWNSTREAM monotonic flags +
  deduped session_revenue."
columns:
  - name: session_key
    data_tests: [not_null, unique]
  - name: reached_view_item
    data_tests: [not_null]
  - name: reached_purchase
    data_tests: [not_null]
  - name: session_revenue
    data_tests:
      - dbt_utils.accepted_range: {min_value: 0}

```

4.4 Keystone test budget

Both keystones **fail silently**: a sessionization bug (wrong key, first-touch leakage, a dropped landing page) or a monotonicity bug (a non-downstream flag, a double-counted purchase) produces no error — just a number that is quietly wrong and corrupts every mart, every metric, and ultimately every agent diagnosis. Structural tests (`not_null` , `unique` , `accepted_values`) catch shape, not *correctness*. Therefore the keystone test budget is spent **golden-value tests first**: `dbt unit_tests` that seed a handful of synthetic events and assert the *exact* derived output — e.g. two events with the same (`user_pseudo_id` , `ga_session_id`) collapse to one session with the earliest `page_location` as `landing_page` ; a returning user's session takes its `event_params` source/medium, not `traffic_source` ; a session whose only funnel event is `purchase` still sets every `reached_*` flag true (monotonicity); a duplicated `purchase` event yields `session_revenue` counted once. These are backed by the singular tests `assert_funnel_monotonicity` (asserts `sessions ≥ reached_view_item ≥ ... ≥ reached_purchase` holds at every grain) and `assert_session_conversion_rate_bounds` (asserts `purchasing_sessions / sessions` stays within `[0, 1]`). The full keystone testing strategy — unit-test `given / expect` fixtures, the singular tests, and the reconciliation backbone — is specified in **section 6 (Testing)**; this section only flags *that* these two models earn the deepest test investment in the project.